

Dependency Injection Lifecycle Container

Dependency Injection (DI) is a core architectural pattern used to decouple high-level modules from low-level service implementations. A "DI Container" manages the lifecycle and resolution of these dependencies, ensuring that components receive their required collaborators at runtime without hardcoded instantiations.

Implement a `DIContainer` class that handles the registration and automated resolution of system services. The container must distinguish between **Transient** services (new instance per call) and **Singleton** services (cached instance reused for all calls), while providing robust error handling for missing service definitions.

Example 1 Input:

```
REGISTER db true RESOLVE db d1 RESOLVE db d2 CHECK_IDENTITY d1 d2
```

Output:

```
Resolved db as d1 Resolved db as d2 True
```

Explanation: The `db` service is registered as a singleton. Both `d1` and `d2` resolutions point to the exact same object instance in memory.

Example 2 Input:

```
REGISTER logger false RESOLVE logger l1 RESOLVE logger l2  
CHECK_IDENTITY l1 l2
```

Output:

```
Resolved logger as l1 Resolved logger as l2 False
```

Explanation: The `logger` service is registered as transient. Each resolution triggers the factory function to create a unique, independent instance.

Function Parameter:

- `service_name` (str): Unique identifier for the service.

- `factory_func` (callable): Function responsible for object creation.
- `singleton` (bool): Flag to enable instance caching.

Returns

- Any: The resolved service instance.

Constraints

- Raise `KeyError("Service <name> not found")` for unregistered lookups.
- The `clear()` method must purge both registrations and cached singletons.
- Factory functions are assumed to be parameterless for this implementation.

Input Format for Custom Testing Input from stdin contains commands: `REGISTER <name> <is_singleton>`, `RESOLVE <name> <var_id>`, `CHECK_IDENTITY <var1> <var2>`, or `CLEAR`.

Sample Case 0 Sample Input

```
REGISTER api true RESOLVE api a1 RESOLVE api a2 CHECK_IDENTITY a1 a2
```

Sample Output

```
Resolved api as a1 Resolved api as a2 True
```

Explanation The API service is cached as a singleton, maintaining state across multiple resolution requests.